

ICML 2024 | Making Semidefinite Programming Actually Scale: USBS Turns Billion-Variable SDPs Into a Usable Tool

USBS, a unified spectral bundle method from UMass Amherst, supports warm-starting, scales past 10^{13} decision variables, and runs roughly 500× faster than the previous state-of-the-art solver

Semidefinite programming (SDP) is one of the most versatile tools in optimization. Combinatorial problems like MaxCut and the quadratic assignment problem, as well as tasks in neural-network verification and control, can all be relaxed into an SDP and solved. The catch is a long-standing one: SDPs are expensive. Classical solvers project onto the semidefinite cone at every iteration, and that projection requires a full eigendecomposition whose cost grows cubically with problem size. So in practice, SDPs tend to get filed under "elegant in theory, intractable at real-world scale."

Recent sketching-based methods, such as CGAL, pushed that ceiling much higher by keeping a low-rank sketch instead of the full matrix. But they pay for the scale in two ways. First, as the problem grows they need more and more iterations, so convergence slows down. Second, and more subtly, they rely on iteration-dependent parameter schedules — and those schedules are exactly what blocks effective warm-starting, the ability to initialize from a previous solution to a closely related problem. That matters, because in many real settings you never solve a single SDP in isolation: data arrives incrementally, or you solve a sequence of nearly identical subproblems inside a mixed-integer or interactive loop. In all of these settings, warm-starting is exactly where the speedup ought to come from, provided the solver can actually exploit it.

This ICML 2024 paper introduces USBS (Unified Spectral Bundling with Sketching): a provably convergent solver that handles both equality and inequality constraints, reliably exploits warm starts, and — with optional matrix sketching — scales to enormous instances while keeping convergence fast. On a MaxCut instance with more than 2×10^9 decision variables, USBS runs about 500× faster than the previous state-of-the-art scalable solver, and warm-starting alone can accelerate convergence by more than 100×.

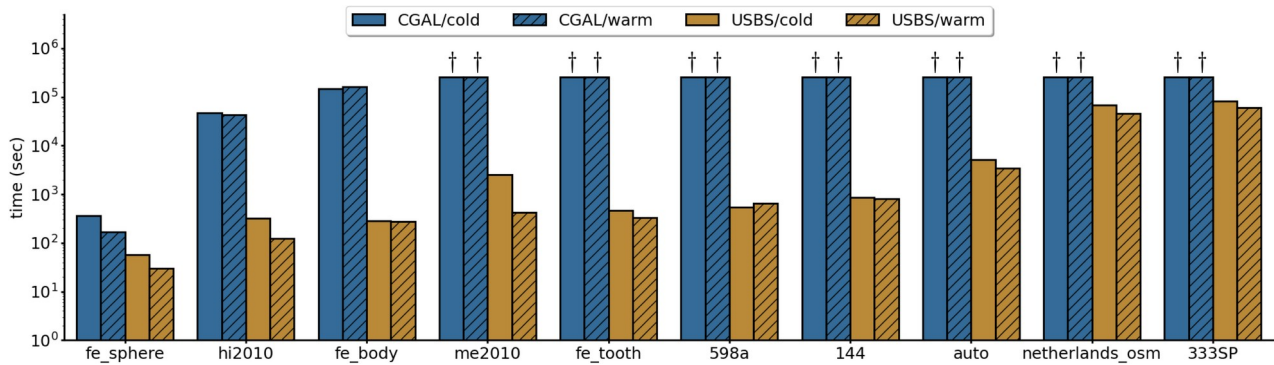


Figure 1. Wall-clock time to reach a moderate accuracy ($\epsilon=10^{-1}$; lower is better) on ten DIMACS10 MaxCut instances, comparing CGAL and USBS with and without warm-starting (built from 99% of the data). Instances are sorted by vertex count n , from 16K to 3.7M. Bars marked † did not reach an ϵ -approximate solution within 72 hours. CGAL fails on 7 of the 10 instances, while USBS solves all ten within 28 hours even without a warm start.

Paper: <https://arxiv.org/abs/2312.11801> | Code: <https://github.com/rangell/usbs> (a standalone pure-JAX implementation that runs on CPU, GPU, and TPU)

The bottleneck: expensive projections, and sketching that blocks warm starts

To see why USBS helps, it is worth pinning down where each generation of solvers gets stuck. For classical interior-point and first-order methods, the dominant cost is the projection onto the semidefinite cone: effectively a full eigendecomposition of an $n \times n$ matrix, which scales cubically in n . Once n reaches the millions, that single step becomes infeasible — which is the concrete technical reason behind the "SDPs don't scale" reputation.

Sketching methods take a different route. Instead of storing the full matrix, they maintain only a low-rank sketch, which brings memory and per-iteration cost down and lets them scale much further. But they have two weak spots. The number of iterations needed to converge grows with problem size, so they slow down as problems get larger; and to guarantee convergence they use iteration-dependent step-size and penalty schedules. Warm-starting wants to jump to a near-optimal initial point and keep going, and that goal is fundamentally at odds with a fixed, iteration-indexed schedule. The net effect: sketching methods scale, but struggle to actually cash in on a warm start.

What USBS contributes: unify the spectral bundle method, and make warm-starting work

Spectral bundle methods are an appealing framework to begin with — low per-iteration cost and fast empirical convergence. But prior versions handled either only equality constraints or only inequality constraints, and none had an efficient standalone implementation that could be evaluated on genuinely massive SDPs. USBS closes those gaps. It is a unified spectral bundle method for general SDPs with both equality and inequality constraints, and it brings three properties together: it reliably leverages warm-start initializations; it can be augmented with

optional matrix sketching that sharply improves scalability while keeping convergence fast; and it comes with provable non-asymptotic convergence rates. The method also exposes model parameters that let the user trade per-iteration complexity against convergence speed, and it ships as a standalone pure-JAX implementation for CPU, GPU, and TPU.

How it works: recast the dual SDP as something cheap to model

USBS starts from the dual semidefinite program and rewrites it as a single unconstrained "penalized dual" objective, combining the largest eigenvalue of $C - \mathcal{A}^*y$, the linear term $\langle b, y \rangle$, and an indicator for the inequality constraints into one function: $f(y) := \alpha \cdot [\lambda_{\max}(C - \mathcal{A}^*y)]_+ + \langle b, y \rangle + \iota_{\mathcal{Y}}(y)$. It then minimizes this objective with a proximal bundle method.

The key move is that each iteration does not optimize the true objective directly. Instead it builds a cheap lower model $\hat{f}_t$ that lives in a low-dimensional subspace — spanned by the current maximum eigenvectors (k_c of them) plus a few directions carrying past spectral information (k_p of them). USBS takes a proximal step against this model, solving a small minimax problem for the candidate iterate $\tilde{y}_{t+1} = \operatorname{argmin}_y \hat{f}_t(y) + (\rho/2)\|y - y_t\|^2$. Then it runs a descent test: if the true objective decreases by at least a fixed fraction β of what the model predicted — that is, $\beta \cdot (f(y_t) - \hat{f}_t(\tilde{y}_{t+1})) \leq f(y_t) - f(\tilde{y}_{t+1})$ — it accepts the step (a descent step); otherwise it takes a null step but still refines the model with the new information. Because only a small set of eigenvectors and, optionally, a low-rank sketch of the primal matrix are ever stored, the whole procedure scales to enormous problems.

That lightweight state is worth pausing on, because it is exactly what makes warm-starting practical. A concrete example comes from interactive entity resolution, where a system resolves author names by solving a growing sequence of closely related SDPs — one more constraint at a time — as shown in the figure below.

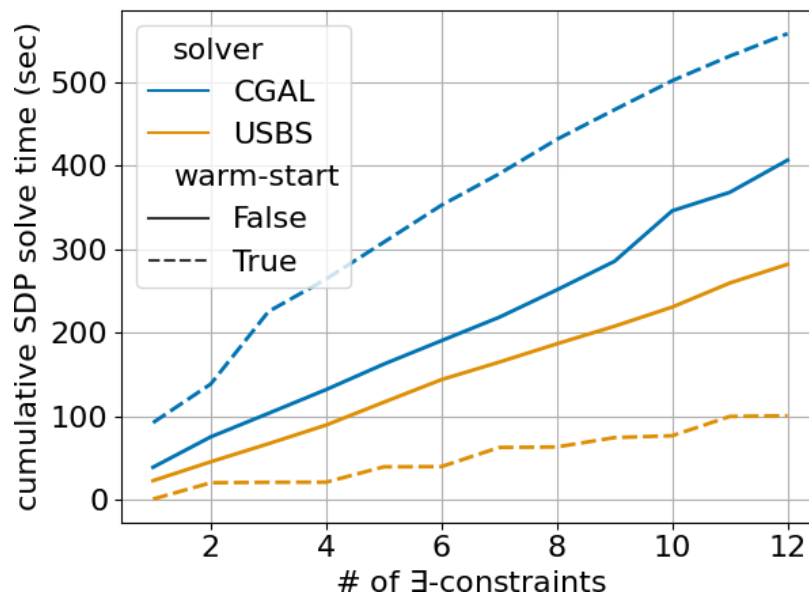


Figure 6. Cumulative SDP solve time (seconds; lower is better) as $\exists$ -constraints are added one at a time in author-coreference resolution. Each new constraint means solving an SDP almost identical to the previous one, so the previous solution can serve as a warm start (dashed lines). USBS (orange) is markedly faster warm-

started than cold, and faster than CGAL (blue) throughout; the gap in USBS's favor widens as the problem grows.

This structure is also why USBS is naturally suited to warm-starting: its state is the current solution y plus a small set of spectral directions, not a schedule welded to an iteration counter, so continuing from the solution of a previous, closely related problem is the natural thing to do. The entity-resolution figure above makes that concrete: each new $\exists$ -constraint only nudges the problem, and USBS resumes from the previous solution instead of restarting.

Experiments: three problem families, one consistent story

USBS is evaluated on three very different problem families. First, MaxCut on ten DIMACS10 graphs with n from 16K to 3.7M, the largest (333SP) yielding more than 10^{13} decision variables. Second, the quadratic assignment problem (QAP) on QAPLIB and TSPLIB instances, whose SDP relaxation has on the order of $O(n^4)$ decision variables — reaching about 1.5×10^9 at $n=198$. Third, interactive entity resolution on three author-coreference datasets (PubMed, QIAN, and SCAD-zbMATH). In every case, a warm start is constructed naturally from a slightly smaller, closely related problem.

The MaxCut story is Figure 1 above: even cold-started, USBS solves all ten instances within 28 hours, while CGAL fails to reach the target accuracy on 7 of them within 72 hours, and on the 2×10^9 -variable instance USBS is about $500\times$ faster. The next two figures turn to QAP — first the convergence dynamics on a single instance, then the aggregate picture across ten instances.

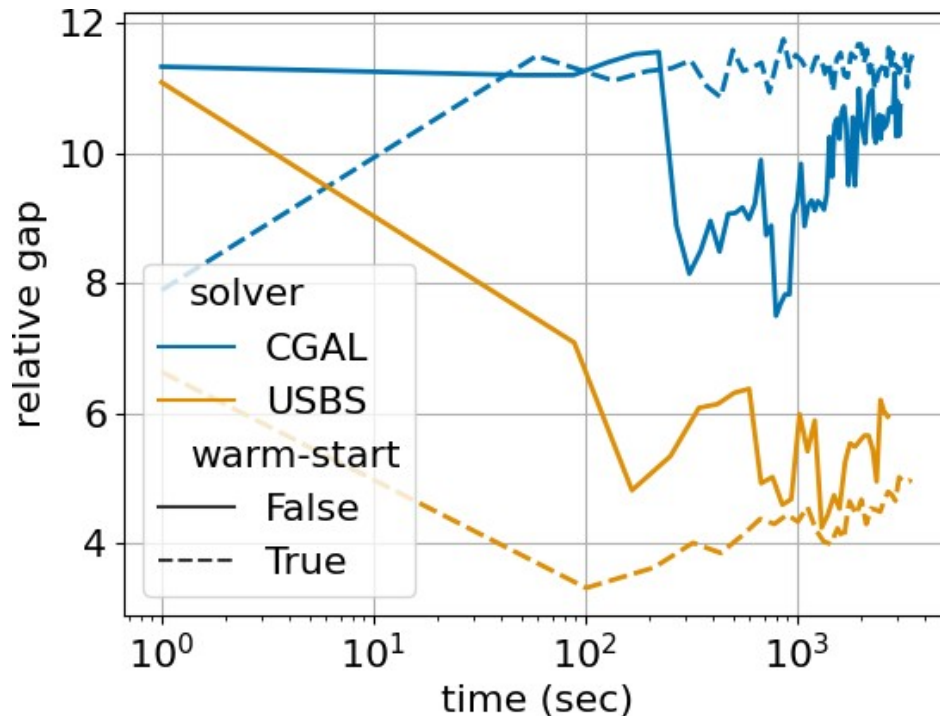


Figure 2. Relative gap (lower is better) versus time over one hour on the QAP instance pr144 ($n=144$). Solid lines are cold-started, dashed lines warm-started. USBS (orange) converges to a far smaller gap than CGAL (blue) whether cold or warm, and clearly descends faster from a warm start; both algorithms find their best rounded solution early in optimization.

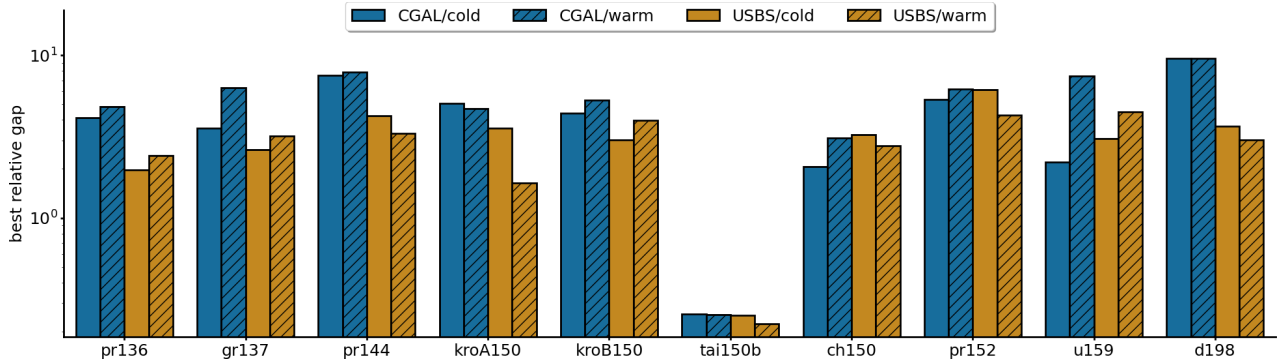


Figure 5. Best relative gap (lower is better) reached within one hour on ten QAPLIB and TSPLIB instances. On most instances USBS (orange) beats CGAL (blue), and warm-starting helps USBS reach a smaller gap still; CGAL is much less reliable at exploiting a warm start.

The numbers that matter most are collected in the table below, side by side for the two generations of solvers.

Table 1. USBS versus the previous scalable solver, CGAL, on the headline metrics.

Metric	CGAL (prior SOTA)	USBS (this paper)
MaxCut instances solved (DIMACS10, 10 total)	3 / 10 (within 72 h)	10 / 10 (≤ 28 h)
Relative speed on the 2×10^9 + variable MaxCut instance	baseline (reference)	$\approx 500\times$ faster
Convergence speedup from warm start vs. cold	often cannot exploit reliably	$> 100\times$
Scale reached (decision variables)	—	$> 10^{13}$ (333SP, $n \approx 3.7M$)
Largest QAP relaxation solved	—	$n=198$, $\approx 1.5 \times 10^9$ variables

Ablations: warm-starting buys 100×+, and kc matters more than kp

Two ablations are especially informative. The first is warm-starting: for USBS, initializing from the previous solution can speed up convergence by more than 100× compared with cold-starting — and, importantly, USBS actually realizes that benefit while CGAL usually cannot. The second concerns the model parameters. The number of current eigenvectors, k_c , matters a great deal, and larger values generally give better convergence — a contrast with the original Helmberg–Rendl spectral bundle method, which fixed k_c to 1. The number of past spectral directions, k_p , turns out to help little and can even hurt. The recommended defaults reflect this, keeping k_c large and k_p small ($r=10$, $p=0.01$, $\beta=0.25$, $k_c=10$, $k_p=1$).

Takeaway, and the honest boundary

The message of USBS is straightforward: large-scale semidefinite programming does not have to be slow, and it does not have to be restricted to a narrow class of problems. By unifying equality and inequality constraints in a single spectral bundle method, adding optional sketching for scalability, and making warm-starting actually work, USBS turns SDPs that were previously considered intractable into a practical tool — particularly well suited to pipelines where data arrives incrementally or where closely related subproblems are solved repeatedly inside a

mixed-integer or interactive loop. The accompanying pure-JAX implementation runs on CPU, GPU, and TPU, which lowers the barrier to adoption further.

The boundaries are worth keeping in mind too. These results are established on three families — MaxCut, QAP, and entity resolution — and the benefit of warm-starting depends on consecutive problems being genuinely close. The best values of k_c and k_p are problem- and hardware-dependent. Even so, USBS meaningfully advances both the scale of SDPs that can be solved and how usable they are in practice.

Concretely, the settings where USBS should help most are the ones where SDPs are solved in sequence rather than once: branch-and-bound for mixed-integer programs, streaming or incrementally arriving data, and interactive systems that refine an answer as constraints accumulate. In each of these, the previous solve is a near-ideal starting point, and USBS is built to use it. For the cases that are simply too large to touch otherwise, the sketching option is the lever that trades a little accuracy for the ability to run at a scale earlier solvers could not reach — which is what lets the method report results on graphs with millions of vertices at all.

Resources — Paper: <https://arxiv.org/abs/2312.11801> · Code (pure JAX, CPU/GPU/TPU): <https://github.com/rangell/usbs>